

Class #3

02.02 Design Quality, Distributed Systems & Observability

Software Architectures · Master in Informatics Engineering

Cláudio Teixeira · claudio@ua.pt

Agenda

- **Block 1 – Design Quality at the Module Level** 50 min
 - Cohesion and Separation of Concerns
 - SOLID – as they emerge from fixing real code
- **Block 2 – Crossing Process Boundaries** 80 min
 - Choreography vs Orchestration
 - Security checkpoint
 - Group exercise
- **Block 3 – Observability** 30 min
 - Three pillars + telemetry data path
 - Redaction layers, policy decisions, privacy tension

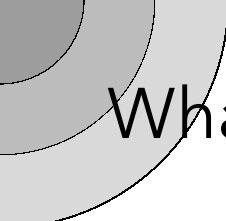
Block 1

Design Quality at the Module Level

What's wrong with this code?

```
public class RocketTicketSaleService
{
    public async Task<SaleResult> ProcessSale(SaleRequest request)
    {
        decimal price = request.PromoCode == "EARLY20" ? 71.99m : 89.99m;
        var seat = _db.Seats.FirstOrDefault(s => s.EventId == request.EventId && !s.IsReserved);
        if (seat == null)
            return new SaleResult { Success = false, Reason = "No seats available" };
        seat.IsReserved = true; seat.UserId = request.UserId; _db.SaveChanges();
        var payment = await _httpClient.PostAsJsonAsync(
            "https://payments.external.com/charge",
            new { amount = price, card = request.CardToken, currency = "EUR" });
        if (!payment.IsSuccessStatusCode)
        {
            seat.IsReserved = false; _db.SaveChanges(); // manual rollback
            return new SaleResult { Success = false, Reason = "Payment failed" };
        }
        await _smtpClient.SendMailAsync(new MailMessage(
            "tickets@rockettickets.com", request.UserEmail,
            "Your ticket!", $"Seat {seat.SeatNumber} - Total: {price:C}"));
        return new SaleResult { Success = true, SeatNumber = seat.SeatNumber };
    }
}
```

This class is in production. It works. **What's the problem?**



What this class is actually doing

One method mixing six responsibilities:

- Logging
- Discount calculation
- Seat reservation
- Manual transaction rollback
- Payment (*via external HTTP call*)
- Email notification

Different teams, same file:

- Marketing changes pricing
- Finance changes providers
- Ops changes logging
- Comms changes email templates

One reason to open it: *anything*. That's the design smell.

Cohesion – the diagnostic tool

"Are these things together because they belong together, or because someone put them here?"

Band	Levels	What it looks like
Bad	Coincidental, Logical, Temporal	Things grouped by accident, category, or timing
Acceptable	Procedural	The sequence justifies the grouping
Good	Communicational, Sequential, Functional	Everything serves one well-defined purpose

`RocketTicketSaleService` is in the **bad** band.

Not because it's buggy. Because it *will be*.



The cost of bad cohesion

A new payment provider is needed. You open `RocketTicketSaleService` .

You need to change the discount logic for loyalty points. You open `RocketTicketSaleService` .

The email template needs updating. You open `RocketTicketSaleService` .

There's a bug in the seat reservation under concurrent load. You open `RocketTicketSaleService` .

Every change risks every other responsibility. Every test must set up the whole world. Every developer needs to understand everything to change anything.

Separation of Concerns – the first cut

// ✅ Each class has one reason to exist

```
public class SeatReservationService
{
    public async Task<Seat?> ReserveAsync(Guid eventId, Guid userId) { ... }
    public async Task ReleaseAsync(Guid seatId) { ... }
}

public class DiscountCalculator
{
    public decimal Apply(decimal basePrice, string? promoCode) { ... }
}

public class NotificationService
{
    public async Task SendConfirmationAsync(string email, Seat seat, decimal amount) { ... }
}
```

We've separated concerns. But `RocketTicketSaleService` still calls `PaymentService` directly. What happens when we need a second payment provider?

Single Responsibility Principle

A class should have only one reason to change.

The violation isn't just structural – it's about **who triggers the change**.

Class	Reason to change
SeatReservationService	Inventory team changes business rules
DiscountCalculator	Marketing changes pricing logic
NotificationService	Comms team changes email templates
PaymentService	Finance changes payment providers

When teams change independently, their code should too.

SRP isn't about lines of code. It's about **organisational coupling**.

Open/Closed Principle – adding a payment provider

// ❌ Without OCP – modify existing code for every new provider

```
public class PaymentService
{
    public async Task<PaymentResult> ChargeAsync(decimal amount, string cardToken)
    {
        if (_config.Provider == "Stripe") return await ChargeStripe(amount, cardToken);
        else if (_config.Provider == "PayPal") return await ChargePayPal(amount, cardToken);
        // every new provider = a risky modification to existing, tested code
    }
}
```

// ✅ With OCP – extend without modifying

```
public interface IPaymentProvider
{
    Task<PaymentResult> ChargeAsync(decimal amount, string cardToken);
}

public class StripePaymentProvider : IPaymentProvider { ... }
public class PayPalPaymentProvider : IPaymentProvider { ... } // new class, zero changes elsewhere
```

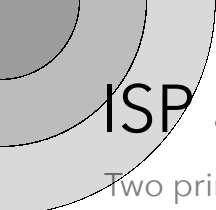
Dependency Inversion Principle

High-level modules depend on contracts, not implementations.

```
// ❌ Hardwired implementation
public class PurchaseOrchestrator
{
    private readonly StripePaymentProvider _payment =
        new StripePaymentProvider();
}
```

```
// ✅ Inject abstraction
public class PurchaseOrchestrator
{
    private readonly IPaymentProvider _payment;
    public PurchaseOrchestrator(IPaymentProvider payment)
        ⇒ _payment = payment;
}
```

Swappable provider. Testable orchestrator. No caller changes when Stripe becomes PayPal.



ISP and LSP – a reference

Two principles that protect the abstractions you just introduced.

Interface Segregation Principle

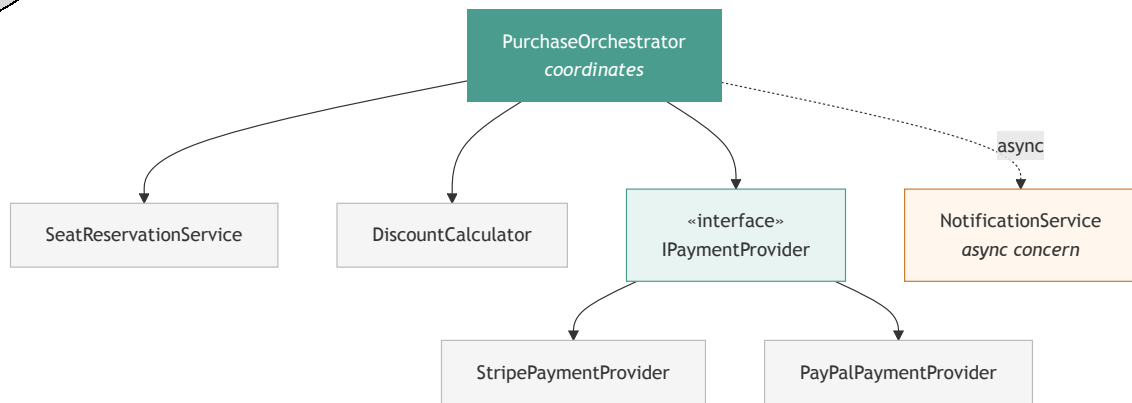
Don't force a class to depend on methods it doesn't use. An `ITicketService` exposing both `Generate()` and `Refund()` forces every consumer to know about refunds – even if they only generate tickets.

Liskov Substitution Principle

Subtypes must honour the contract of their base type. A `PayPalPaymentProvider` that throws `NotImplementedException` on `ChargeAsync` is an LSP violation that breaks every caller that trusted `IPaymentProvider`.

Both matter most at service boundaries – which is exactly where we're going next.

The refactored module boundary



Each box: one reason to change. One team. One test surface.

Before we cross the boundary

"Which of these changes would have been hardest to make at 3am during a live flash sale incident?"

God class version: you'd need to understand everything to change anything safely.

Refactored version: find the right class in 30 seconds, change it, know exactly what you *haven't* broken.

That's the architectural value of cohesion and SOLID. Not elegance. Not theory. **Operational confidence under pressure.**

Your payment provider just moved to another service. Everything you just built now has to talk across a network.



Revisit

Block 1 – Home Reading

Definitions, full taxonomies, and further references for the topics covered in Block 1. Not presented live – acknowledged and pointed to.



Revisit – Terminology

Cohesion – the degree to which the elements inside a module belong together. High cohesion means a module has a single, well-defined purpose; everything inside it relates to that purpose and nothing else.

Coupling – the degree of direct knowledge one component has of another. Introduced in Class 2. Here we extend it: loose coupling at the class level is the foundation for loose coupling at the service level.

Separation of Concerns (SoC) – a design principle for dividing a program into distinct sections, each addressing a separate concern. A concern is any set of information that affects the code of a program.

SOLID – Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. Five principles for object-oriented design that collectively manage how dependencies form and propagate through a codebase.

Law of Demeter (LoD) / Principle of Least Knowledge – an object should only call methods of: itself, objects passed as parameters, objects it creates, and objects it directly holds. It should not reach through other objects to access their dependencies.

Abstraction – in the context of DIP, an interface or abstract class that defines a contract without specifying implementation. Both high-level and low-level modules depend on the abstraction rather than on each other.



Revisit – Law of Demeter (LoD)

Principle of Least Knowledge

An object should only call methods of: itself, objects passed as parameters, objects it creates, and its direct component objects.

The violation is called a **"train wreck"** – a chain of calls that reaches through multiple object levels.

```
// ❌ Violation – PurchaseOrchestrator navigates through request internals
public class PurchaseOrchestrator {
    public void ValidateOrder(SaleRequest request) {
        string city = request.Customer.Address.City; // train wreck
        // PurchaseOrchestrator now depends on Customer, Address, and City
        // A rename in Address breaks PurchaseOrchestrator
    }
}

// ✅ Applying LoD – ask SaleRequest for what you need directly
public class PurchaseOrchestrator {
    public void ValidateOrder(SaleRequest request) {
        string city = request.GetCustomerCity();
        // Only depends on SaleRequest – internal navigation is encapsulated
    }
}
```

Architectural consequence: LoD limits the blast radius of structural changes. If `Address` gains a new layer of nesting, only `SaleRequest` needs updating – not every caller that navigated through it. In distributed systems, LoD at the API level means consumers should never need to join across services to get what they need from a single call.



Revisit – Aim for Loose Coupling

Coupling measures how much one component knows about another. Tight coupling means a change in one component forces changes in others. Loose coupling limits that blast radius.

At the class level, loose coupling is achieved through:

- **Abstractions (interfaces)** – depend on `IPaymentProvider`, not `StripePaymentProvider`
- **Dependency injection** – receive dependencies rather than constructing them
- **Events** – communicate through published events rather than direct method calls

```
// ❌ Tight coupling – PurchaseOrchestrator cannot exist without StripePaymentProvider
```

```
public class PurchaseOrchestrator {  
    private readonly StripePaymentProvider _stripe = new StripePaymentProvider();  
    // Adding PayPal = modify this class  
    // Testing = real Stripe connection required  
}
```

```
// ✅ Loose coupling – depends only on the abstraction
```

```
public class PurchaseOrchestrator {  
    private readonly IPaymentProvider _payment;  
    public PurchaseOrchestrator(IPaymentProvider payment) ⇒ _payment = payment;  
    // Adding PayPal = new class, zero changes here  
    // Testing = inject a test double, no network required  
}
```

Architectural consequence: Loose coupling at the class level is the prerequisite for loose coupling at the service level. You cannot have loosely coupled microservices if the classes inside them are tightly coupled to each other.



Revisit – Get High Cohesion

Cohesion measures how strongly related the elements inside a module are. A highly cohesive module does one thing well. A low-cohesion module does many things – some of which have nothing to do with each other.

The test for functional cohesion: *"Can I describe what this class does in one sentence without using the word 'and'?"*

If the answer is "it reserves seats **and** charges payments **and** sends emails" – cohesion is low.

```
// ❌ Low cohesion – SeatManager mixes three unrelated domains
public class SeatManager {
    public Seat? Reserve(Guid eventId, Guid userId) { ... } // inventory
    public void SendConfirmation(string email) { ... } // notification
    public decimal CalculateDiscount(string promoCode) { ... } // pricing
}

// ✅ High cohesion – SeatReservationService does exactly one thing
public class SeatReservationService {
    public async Task<Seat?> ReserveAsync(Guid eventId, Guid userId) { ... }
    public async Task ReleaseAsync(Guid seatId) { ... }
    public bool IsAvailable(Guid eventId) { ... }
}

// Every method serves one purpose: managing seat reservations
```

Architectural consequence: High cohesion and loose coupling are complementary goals. High cohesion gives each module a single reason to exist; loose coupling ensures that reason does not drag other modules into every change.



Revisit – Cohesion Types (1 of 7)

Coincidental – the least desirable

Definition: A random collection of unrelated functions within the same module. There is no logical, temporal, or functional relationship between elements – they are together by accident.

```
// ❌ Coincidental – OpenWindow, SendEmail, WriteToFile share nothing
public class Miscellaneous {
    public void OpenWindow()           { /* UI logic */      }
    public void SendEmail(string to)   { /* SMTP logic */   }
    public void WriteToFile(string content) { /* File I/O */  }
}
```

How to improve: Divide into classes with a single domain – `WindowManager`, `EmailSender`, `FileWriter`. Each then has the opportunity to achieve functional cohesion.

Architectural consequence: Coincidental cohesion typically appears in legacy systems as "utility" or "helper" classes that accumulate unrelated operations over time. `RocketTicketSaleService` from Block 1 is coincidentally cohesive – logging, pricing, inventory, payment, and notification ended up together because they were all needed in one transaction.



Revisit – Cohesion Types (2 of 7)

Logical

Definition: Parts are grouped by logical category but are not related by time, sequence, or task. The grouping reflects a mental category, not a behavioural one.

The distinction from coincidental cohesion is subtle: the elements seem related because they are all "utilities" or all "services", but they share no data, no sequence, and no purpose.

```
// ❌ Logical – grouped under "services" but sharing nothing operationally
public class UtilityServices {
    public void LogEvent(string message)           { ... }
    public void SendEmail(string to, string subject, string body) { ... }
    public void ProcessUserInput(string input)      { ... }
}
```

How to improve: Separate by domain responsibility – `EventLogger` , `EmailDispatcher` , `InputProcessor` . The category "utility" is not a justification for co-location.

Architectural consequence: Logical cohesion is common in teams that organise code by layer (controllers, services, repositories) rather than by domain. It makes code easy to find by type but hard to change without side effects.



Revisit – Cohesion Types (3 of 7)

Temporal

Definition: Elements are grouped because they execute at the same time, not because they belong together semantically. Timing is not cohesion.

```
// ❌ Temporal – LogStart/LogEnd are lifecycle events; LogError is fault handling
public class Logger {
    public void LogStart()           ⇒ Console.WriteLine("Start");
    public void LogEnd()             ⇒ Console.WriteLine("End");
    public void LogError(string msg) ⇒ Console.WriteLine($"Error: {msg}");
}
// These share a name prefix and a timing context, not a purpose
```

How to improve: `ProcessLogger` for lifecycle events, `ErrorLogger` for fault handling. Each can then evolve independently as requirements change.

Architectural consequence: Temporal cohesion often appears in initialisation and shutdown routines. It is acceptable at system entry points but should not propagate into domain logic, where it masks the absence of a real design.



Revisit – Cohesion Types (4 of 7)

Procedural – the acceptable threshold

Definition: Elements are grouped because they always execute in a specific sequence. The sequence itself is the justification for co-location. This is the acceptable threshold.

```
// ✓ Procedural – ConfigureSettings must precede SetupDatabase,  
// which must precede StartServices. Sequence is the coherence.  
public class Startup {  
    public void Initialize() {  
        ConfigureSettings();  
        SetupDatabase();  
        StartServices();  
    }  
    private void ConfigureSettings() { ... }  
    private void SetupDatabase() { ... }  
    private void StartServices() { ... }  
}
```

When to improve further: Extract `ConfigurationInitializer`, `DatabaseInitializer`, `ServiceStarter` when each step grows in complexity beyond a few lines. The sequence can still be owned by `Startup`, but each step becomes a separately testable unit.

Architectural consequence: Procedural cohesion is a pragmatic compromise. It becomes a problem when the sequence expands to include steps from different domains – at that point, temporal or coincidental cohesion has crept in under a sequential label.



Revisit – Cohesion Types (5 of 7)

Communicational

Definition: Elements work on the same data or closely related data. The shared data is the justification for co-location. This is clearly better than logical cohesion – the data relationship is real, not categorical.

```
// ✅ Communicational – all methods operate on the same internal data structure
public class ReportGenerator {
    private readonly List<int> _data = new List<int>();

    public void AddData(int value) ⇒ _data.Add(value);
    public int Sum() ⇒ _data.Sum();
    public double Average() ⇒ _data.Average();
    public int Max() ⇒ _data.Max();
}
// The shared data (List<int>) is what makes these methods belong together
```

Architectural consequence: Communicational cohesion is common and generally healthy. It breaks down when "operating on the same data" expands to include operations from different domains – at which point the class needs to be split by function, not by data access.



Revisit – Cohesion Types (6 of 7)

Sequential

Definition: Output from one element directly serves as input to the next. The data pipeline is the justification. This is a stronger relationship than communicational cohesion – the elements form a processing chain, not just a shared data store.

```
// ✅ Sequential – CleanData's output is TransformData's input
public class DataProcessor {
    public string ProcessData(string input) {
        var cleaned = CleanData(input);
        return TransformData(cleaned);
    }
    private string CleanData(string data)    ⇒ data.Trim().ToLower();
    private string TransformData(string data) ⇒ $"[processed] {data}";
}
// The pipeline relationship (clean → transform) justifies the co-location
```

Architectural consequence: Sequential cohesion appears naturally in pipeline architectures (ETL, middleware chains, validation pipelines). It tends to remain healthy as long as each step in the pipeline has a single transformation responsibility. When steps start doing different domain things, refactor into separate processors connected by a pipeline.



Revisit – Cohesion Types (7 of 7)

Functional – the most desirable

Definition: Every element contributes to one single, well-defined task. Nothing inside the module is there for any other reason.

The test: *"Can I describe what this module does in one sentence without using the word 'and'?"*

```
// ✅ Functional – every method serves one purpose: arithmetic
public class Calculator {
    public int Add(int a, int b)      ⇒ a + b;
    public int Subtract(int a, int b) ⇒ a - b;
    public int Multiply(int a, int b) ⇒ a * b;
}

// ✅ In RocketTickets: SeatReservationService does exactly one thing
public class SeatReservationService {
    public async Task<Seat?> ReserveAsync(Guid eventId, Guid userId) { ... }
    public async Task        ReleaseAsync(Guid seatId)                { ... }
    public bool               IsAvailable(Guid eventId)               { ... }
    // One sentence: "manages the reservation state of seats at events"
}
```

Architectural consequence: Functionally cohesive classes are the easiest to test, the easiest to deploy independently, and the easiest to hand to a different team. In distributed systems, each service should aim for functional cohesion at the service boundary – not just inside the codebase.



Revisit – SOLID: S

Single Responsibility Principle

A class should have only one reason to change.

The "reason to change" is organisational, not just structural: different teams, compliance requirements, and deployment cadences are different reasons to change.

```
// ❌ Violation – two reasons to change: authentication logic AND user data persistence
public class UserManager {
    public bool AuthenticateUser(string username, string password) { ... }
    public void SaveUserData(string userData) { ... }
}

// ✅ Each class changes for exactly one reason
public class Authenticator {
    public bool AuthenticateUser(string username, string password) { ... }
}

public class UserDataManager {
    public void SaveUserData(string userData) { ... }
}
```

Architectural consequence: If marketing and finance both modify the same class – one to adjust pricing, the other to meet compliance requirements – SRP is being violated at the organisational level. In RocketTickets, `DiscountCalculator` changes when marketing changes pricing; `PaymentService` changes when finance changes providers. They should never share a class.



Revisit – SOLID: O

Open/Closed Principle

A module should be open for extension, closed for modification.

New behaviour should arrive through new classes or implementations, not edits to stable code.

```
// ❌ Every new type changes existing code
public class ReportService {
    public void Generate(string type) {
        if (type == "Sales")      GenerateSales();
        else if (type == "Inventory") GenerateInventory();
    }
}

// ✅ Extend with new classes
public abstract class Report {
    public abstract void Generate();
}

public class SalesReport : Report { public override void Generate() { ... } }
public class InventoryReport : Report { public override void Generate() { ... } }
```

Architectural consequence: Every `if (_config.Provider == "X")` branch is an OCP smell. In RocketTickets, adding a provider should mean **new class**, not editing `PaymentService`.



Revisit – SOLID: L

Liskov Substitution Principle

Subtypes must preserve the contract that callers rely on.

```
public interface IPaymentProvider {  
    Task<PaymentResult> ChargeAsync(decimal amount, string token);  
}  
  
// ✅ Valid subtype: honours the contract  
public class StripePaymentProvider : IPaymentProvider { ... }  
  
// ❌ Same interface, broken contract  
public class BrokenProvider : IPaymentProvider {  
    public Task<PaymentResult> ChargeAsync(decimal amount, string token)  
        ⇒ throw new NotImplementedException();  
}
```

Architectural consequence: DIP gives replaceability only if LSP holds. A provider injected via `IPaymentProvider` that throws on `ChargeAsync` breaks every caller that trusted the interface.



Revisit – SOLID: I

Interface Segregation Principle

No client should be forced to depend on methods it does not use.

Fat interfaces create unnecessary coupling for implementors and consumers.

```
// ❌ Too broad for all consumers
public interface ITicketService {
    Ticket Generate(Sale sale);
    void Refund(string ticketId);
}

// ✅ Split by capability
public interface ITicketGenerator { Ticket Generate(Sale sale); }
public interface ITicketRefunder { void Refund(string ticketId); }

public class TicketService : ITicketGenerator, ITicketRefunder { ... }
```

Architectural consequence: Split interfaces let teams depend only on the workflows they use. At service boundaries, this is what enables independent evolution.



Revisit – SOLID: D

Dependency Inversion Principle

High-level modules and low-level modules both depend on abstractions.

```
// ❌ Hardwired dependency
public class DataService {
    private readonly DataRepository _repo = new DataRepository();
}

// ✅ Both depend on the abstraction
public interface IDataRepository { void Save(object data); }
public class DataRepository : IDataRepository { public void Save(object data) { ... } }
public class DataService {
    private readonly IDataRepository _repo;
    public DataService(IDataRepository repo) { _repo = repo; }
    public void SaveData(object data) => _repo.Save(data);
}
```

Architectural consequence: DIP enables testability and replaceability. Without it, OCP has nowhere to attach new behaviour (for example, a new payment provider).



Revisit – Further Reading

Books

- **Bass, Clements, Kazman (1998)** – *Software Architecture in Practice*
Modifiability chapters connect to cohesion, SRP, and OCP.
- **Martin (2017)** – *Clean Architecture*
SOLID extended from classes to components and boundaries.

Academic + Course

- **Garlan & Shaw (1993)** – *An Introduction to Software Architecture*
Core vocabulary for architectural structure and reasoning.
- **O'Reilly Playlist**
learning.oreilly.com/playlists/e4bbd2ea-5604-419c-8f40-2b0a06e7f8aa

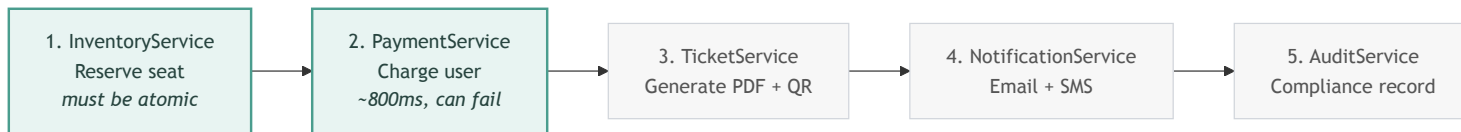
Block 2

Crossing Process Boundaries

"Your module is clean. Now the payment provider lives in another team's service. The same design now has to survive a network that can fail."

The purchase flow

A RocketTickets flash sale purchase triggers **five steps**:

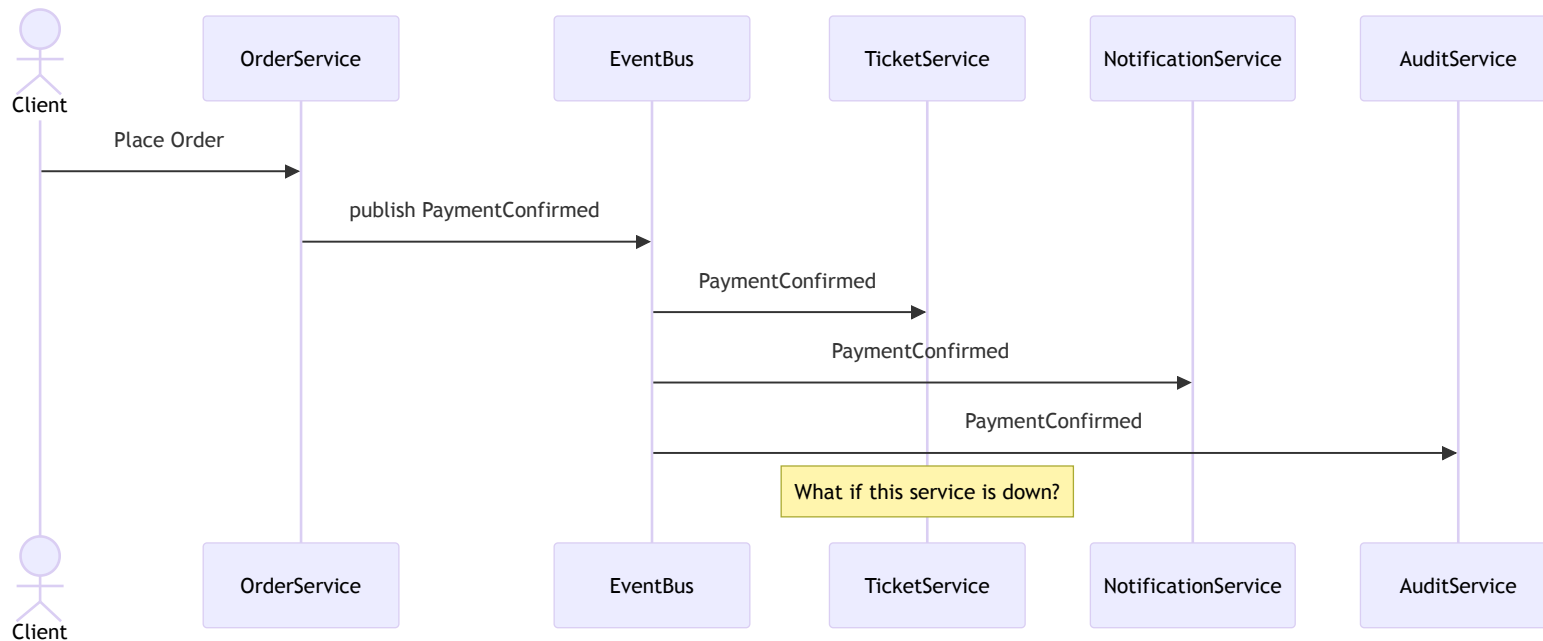


Transactional core: **1-2** · Downstream reactions: **3-5**

Who should be in charge of this flow?

Choreography

Decentralised. Event-driven. Services react to events independently.



Benefits: scalability · loose coupling · no single point of failure

Choreography – the hard question

PaymentService charged the user and emitted `PaymentConfirmed`. TicketService is down. The event is consumed. What happens to the user?

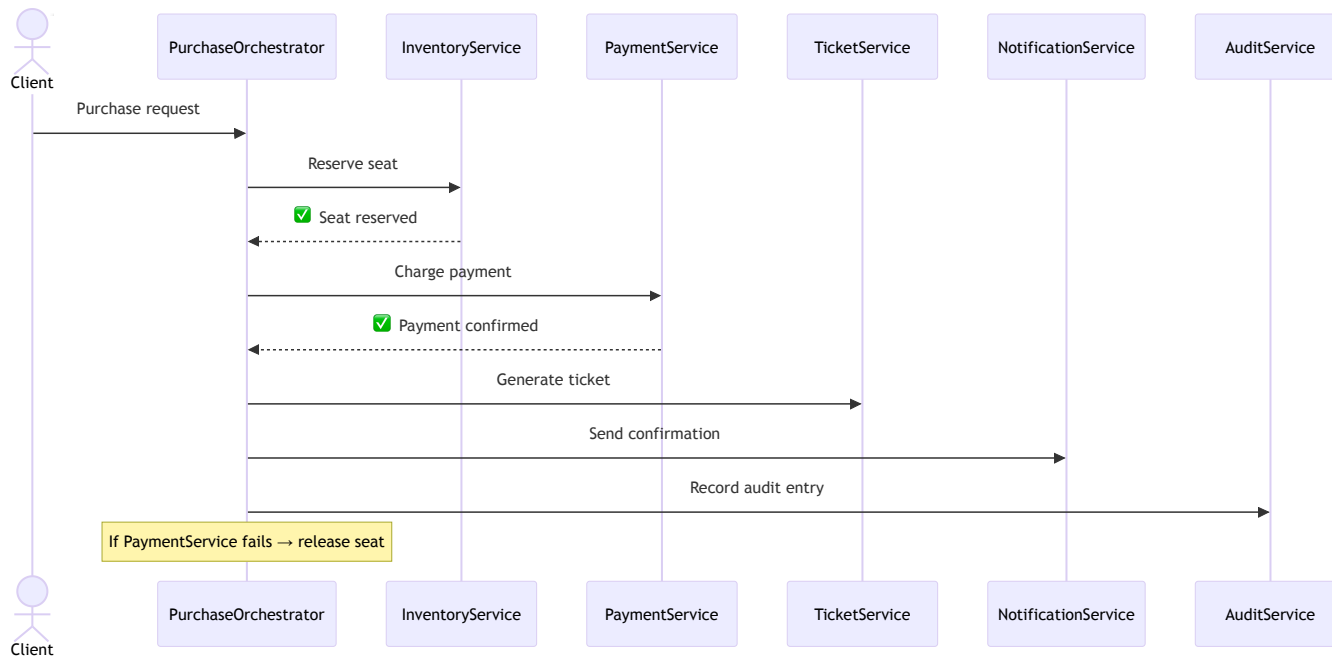
- Who owns the compensation?
- How do you know `TicketService` ever processed the event?
- What's your retry and dead-letter strategy?

Choreography distributes responsibility. That's the strength.

It also distributes accountability. **That's the cost.**

Orchestration

Centralised. Explicit flow. The orchestrator dictates each step.



Benefits: explicit rollback · single place to reason about failures · observable



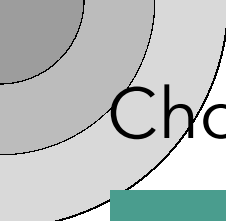
Orchestration – the hard question

50,000 concurrent purchases. Each waiting ~800ms for PaymentService. All routed through one orchestrator. Draw the queue.

- What's the queue depth at second 90 of the flash sale?
- What's your timeout strategy when the orchestrator becomes the bottleneck?
- What happens when the orchestrator itself fails mid-transaction?

Orchestration centralises control. That's the strength.

It also centralises the bottleneck. **That's the cost.**

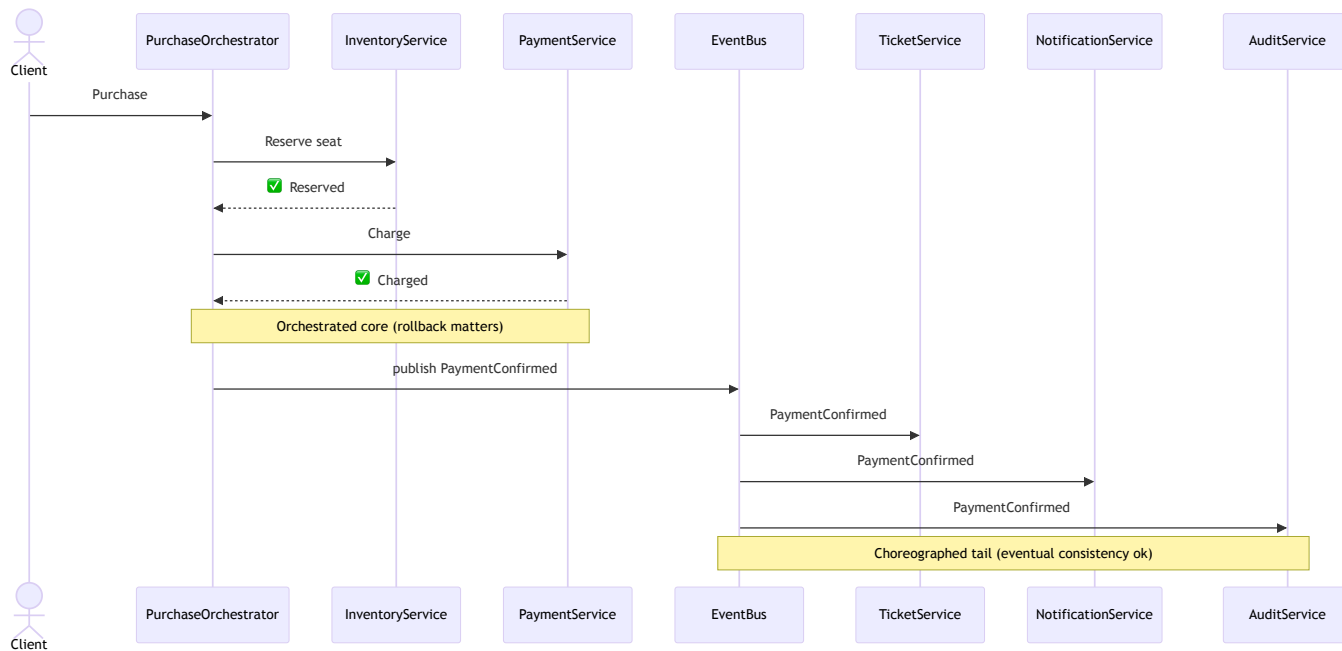


Choreography vs Orchestration

	Choreography	Orchestration
Control	Decentralised	Centralised
Coupling	Low	Potentially higher
Scalability	Highly scalable	Bottleneck risk
Failure handling	Distributed, harder to trace	Centralised, explicit rollback
Observability	Harder – flow is implicit	Easier – flow is explicit
Where it excels	Downstream reactions	Transactional core

The real answer: hybrid

Orchestrate the transactional core. Choreograph the downstream reactions.



The question that cuts through architecture debates: *What are the consistency requirements, and where do they end?*

Security checkpoint



Distributed systems expand your attack surface

Three specific ways crossing a process boundary changes your security picture:

1. Data crosses boundaries

Encryption in transit is not optional. TLS everywhere, including internal service-to-service calls.

2. Services must authenticate each other

Not just users. mTLS, service mesh, per-service API keys. Harder than it sounds at scale.

3. Your observability data itself can become a liability

A trace that captures a POST body. A log line with an email address. A metric tag encoding a user ID. The tools you use to observe your system may be leaking what your system is supposed to protect.

Group Exercise

The Flash Sale Architecture Decision

 **40 minutes**

Setup: 5 min · Group work: 25 min · Presentations: 8 min · Debrief: 7 min

Exercise – Scenario + Questions

RocketTickets expects **50,000 users** in the first 90 seconds. Flow: **Inventory** (atomic) → **Payment** (~800ms, can fail) → **Ticket** → **Notification** → **Audit**

Team Alpha (*Inventory + Payment*)

- Wants a central `PurchaseOrchestrator`
- Prioritises explicit rollback after a past "charged but no ticket" failure

Team Beta (*Ticket + Notification + Audit*)

- Wants choreography on `PaymentConfirmed`
- Prioritises scale and avoiding a flash-sale bottleneck

Your group must answer

1. Which pattern do you recommend, and why?
2. What is the single biggest risk, and how do you mitigate it?
3. What would change your recommendation?

CTO review starts in 20 minutes. **You are the advisory panel.**



Debrief

Neither answer is wrong. The reasoning can be.

The hybrid answer:

Orchestrate the transactional core – everything where "money has moved" requires rollback guarantees.

Choreograph the downstream reactions – eventual consistency is acceptable here.

The question that cuts through any architecture debate: *What are the consistency requirements, and where do they end?*



Revisit

Block 2 – Home Reading

Full definitions, pros and cons, and decision guidance for choreography and orchestration. Not presented live – acknowledged and pointed to.



Revisit – Terminology

Choreography – services react to events independently. No central controller directs the full flow.

Orchestration – a central orchestrator calls each step in sequence and owns failure/compensation logic.

Event Bus – messaging infrastructure for publish/subscribe without direct service-to-service coupling.

Core coordination terms for Block 2.



Revisit – Terminology (2 of 2)

Saga – a sequence of local transactions plus compensating actions to undo earlier work when a later step fails.

Eventual Consistency – replicas converge over time. Appropriate for downstream reactions, not for "money has moved" decisions.

Idempotency – repeating the same request yields the same outcome. Essential for safe retries.

Example: `ReserveAsync` should return the same reservation on retry, not create duplicates.

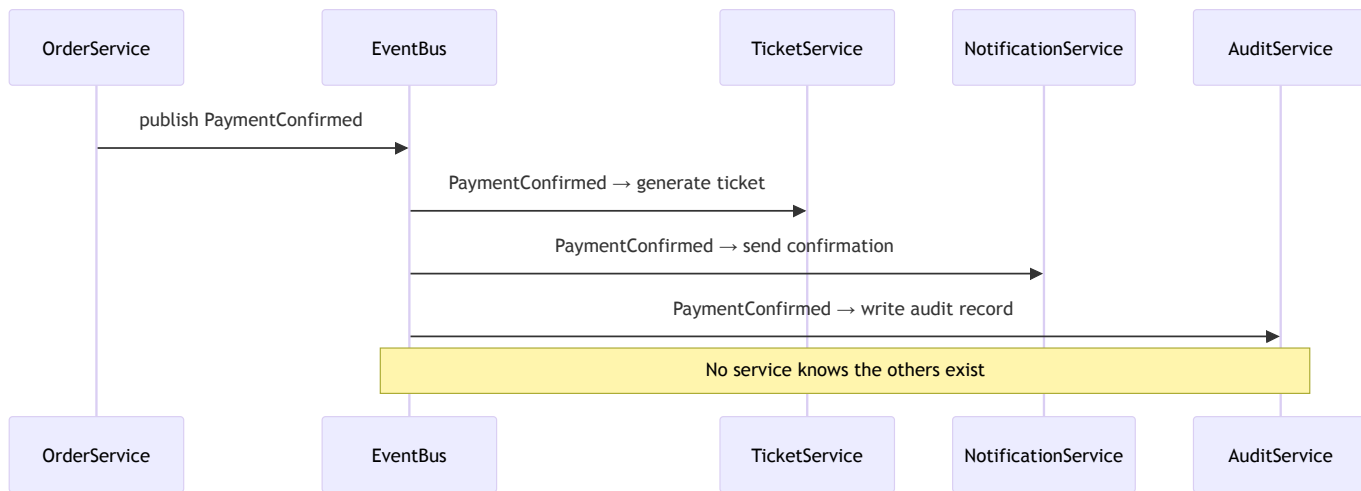


Revisit – Choreography

Definition and Characteristics

Each service publishes events and reacts to the events it cares about. No service owns the full workflow.

How the RocketTickets flash sale purchase looks under choreography:



- OrderService does not need to know who subscribes.
- The workflow emerges from subscriptions, not a central controller.



Revisit – Choreography

Pros and Cons

Advantages

- **Loose coupling** – services know only event types, not each other; teams deploy independently
- **Independent scalability** – each service scales without coordinating with others
- **Resilience** – no central coordinator means no single point of failure in the coordination layer
- **Team autonomy** – teams evolve subscribers independently

Disadvantages

- **Distributed accountability** – understanding the full flow requires reading multiple codebases
- **Hard to trace failures** – reconstructing the event chain when something goes wrong is non-trivial without dedicated tooling (distributed tracing)
- **Compensation complexity** – each service must implement its own compensating actions; there is no central rollback
- **No global view** – no single place to answer "is this purchase complete?"
- **Testing complexity** – end-to-end tests need the bus and subscribers

Best suited for: downstream reactions where eventual consistency is acceptable – ticket generation, notifications, audit records.

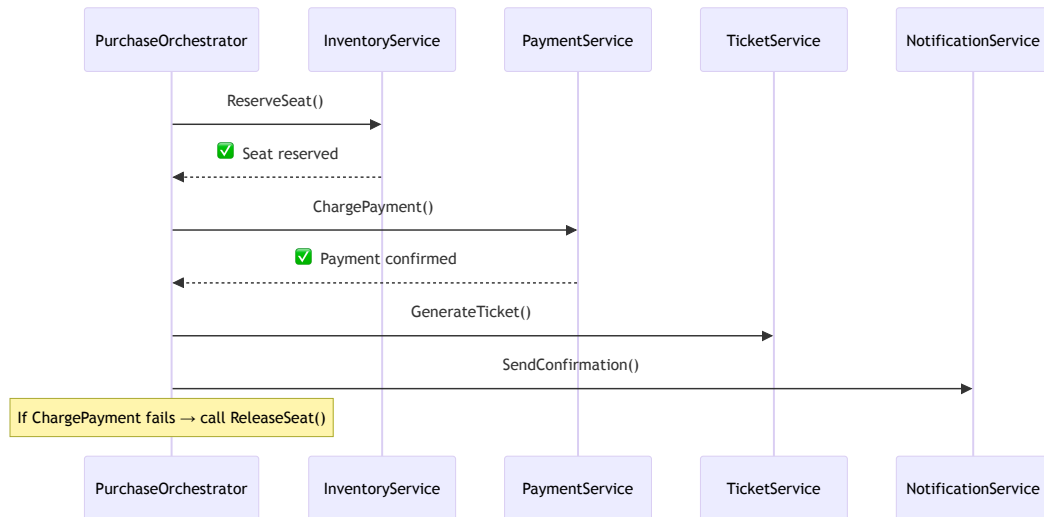


Revisit – Orchestration

Definition and Characteristics

The orchestrator explicitly calls services in sequence, drives the workflow, and handles failures.

How the RocketTickets flash sale purchase looks under orchestration:



- Full flow is visible in `PurchaseOrchestrator`.
- Rollback is explicit, not a distributed compensating event.



Revisit – Orchestration

Pros and Cons

Advantages

- **Single point of reasoning** – the full flow is in one place; a developer can read the orchestrator and understand the whole transaction
- **Explicit rollback** – compensating actions are orchestrated like forward actions; straightforward to reason about and test
- **Easier observability** – the orchestrator provides a natural trace of the entire flow without requiring distributed trace correlation
- **Transactional control** – easier to implement ACID-like guarantees within the orchestrator's scope
- **Idempotency enforcement** – retry logic and deduplication can be managed centrally

Disadvantages

- **Central bottleneck** – the orchestrator processes every request; at 50,000 concurrent purchases each waiting 800ms, the orchestrator is the queue
- **Higher coupling** – the orchestrator must know about every service it coordinates; adding a new step requires modifying the orchestrator
- **Single point of failure** – if the orchestrator fails mid-transaction, recovery requires careful state management
- **Latency accumulation** – every service call adds coordination overhead through the orchestrator
- **Team dependency** – changes to the flow require touching the orchestrator, which may be owned by a different team

Best suited for: the transactional core – anywhere that rollback guarantees and immediate consistency are required.



Revisit – Decision Guide

The practical decision is not "choreography vs orchestration" but: **where does orchestration end and choreography begin?**

Question	Signals orchestration	Signals choreography
Consistency requirement	Must be immediately consistent	Eventual consistency is acceptable
Failure handling	Explicit rollback required	Per-service compensation is acceptable
Flow ownership	One team owns the complete flow	Multiple teams own independent reactions
Scalability pressure	Moderate, predictable load	High, unpredictable concurrent load
Coupling tolerance	Services can know about each other	Services must be fully independent
Audit requirement	Single orchestrator log is sufficient	Requires distributed trace correlation

The heuristic for RocketTickets:

Has money moved, or might money move? Use orchestration. Is this a downstream reaction to a confirmed transaction? Use choreography.

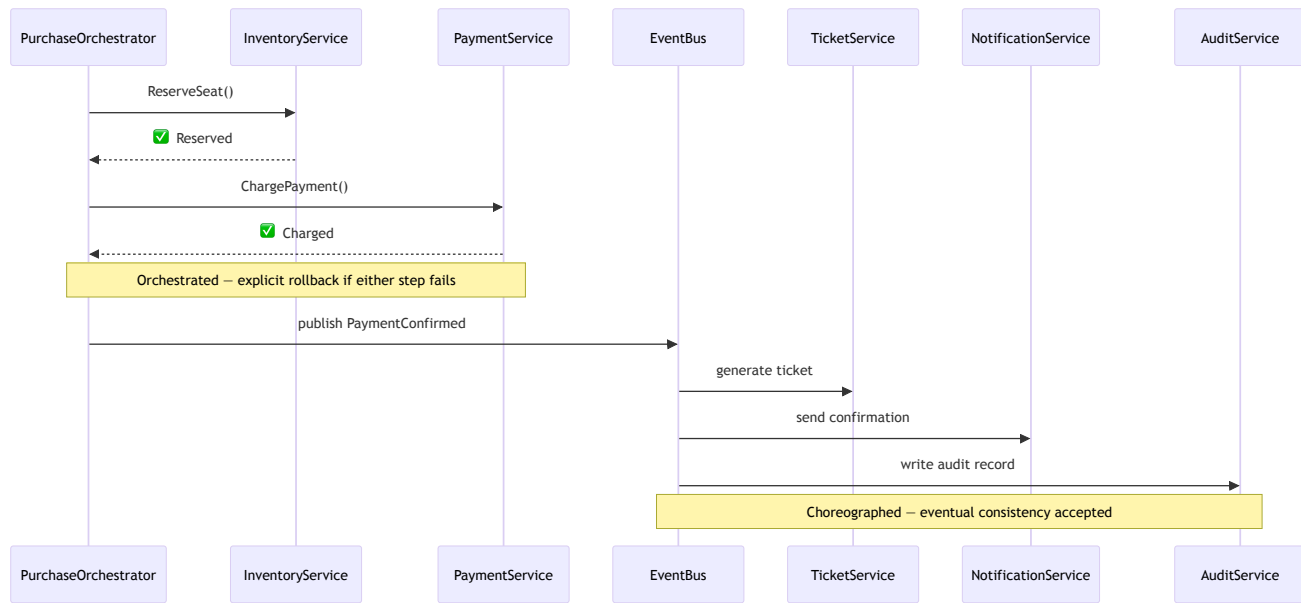
Everything upstream of `PaymentConfirmed` – seat reservation, payment charge – requires orchestration-grade guarantees. Everything downstream – ticket generation, notifications, audit records – can tolerate eventual consistency.



Revisit – The Hybrid Pattern

Most production e-commerce systems use neither pure choreography nor pure orchestration. They combine both – orchestrating the transactional core and choreographing the downstream reactions.

The boundary: drawn at the point where consistency requirements change. In RocketTickets: after `PaymentService` confirms the charge and before the event is published.



The architectural question that cuts through debates:

What are the consistency requirements, and where do they end?



Revisit – References

Books

Bass, L., Clements, P., Kazman, R. (1998). *Software Architecture in Practice*. Addison-Wesley. The availability and modifiability chapters directly address why distributed interaction patterns matter for architectural quality attributes.

Online

Camunda (2023). Orchestration vs Choreography. camunda.com/blog/2023/02/orchestration-vs-choreography Practical framing of the tradeoff from a workflow engine perspective; useful alongside the exercise debrief.

Microsoft (2023). Choreography pattern. learn.microsoft.com/en-us/azure/architecture/patterns/choreography Reference implementation patterns and failure scenarios; read alongside the Camunda piece for the full comparison.

Course

O'Reilly Course Playlist – learning.oreilly.com/playlists/e4bbd2ea-5604-419c-8f40-2b0a06e7f8aa Curated reading for this course. The distributed systems and choreography sections are directly relevant to Block 2 revision.

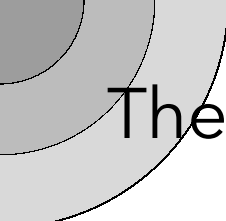


2:10 – 30 min

Block 3

Observability

"You've designed a distributed system. How do you know it's working? And what happens when the data you collect to answer that question contains information you shouldn't have collected?"



The three pillars – where you stand now

Pillar	What it captures	Privacy trap
Logs	Records of events. Structured, searchable, aggregatable.	That email address in the log line is PII
Metrics	Quantitative, time-series. System health + business performance.	<code>user_id=12345</code> as a metric tag is PII in disguise
Traces	Request paths across services. Inter-service latency.	Traces capture request payloads – card tokens, promo codes

Each pillar has a shadow side.

That shadow is the content of Block 3.

The helpful log that violates GDPR

```
// A developer added this to debug a payment issue
_logger.LogInformation(
    "Payment failed for user {Email} with card ending {Last4} – amount {Amount}",
    user.Email,          // ❌ PII – requires documented legal basis under GDPR Art. 6
    card.Last4,          // ❌ PCI-DSS concern
    amount);

// This log is now in your ELK Stack.
// Retained for 90 days.
// Accessible to anyone with log access.
// GDPR says this needs a documented legal basis.
// Do you have one?
```

The developer wasn't malicious. They were debugging. **That's the problem.** The system made it easy to log PII and hard to know you were doing it.

The trace that captures too much

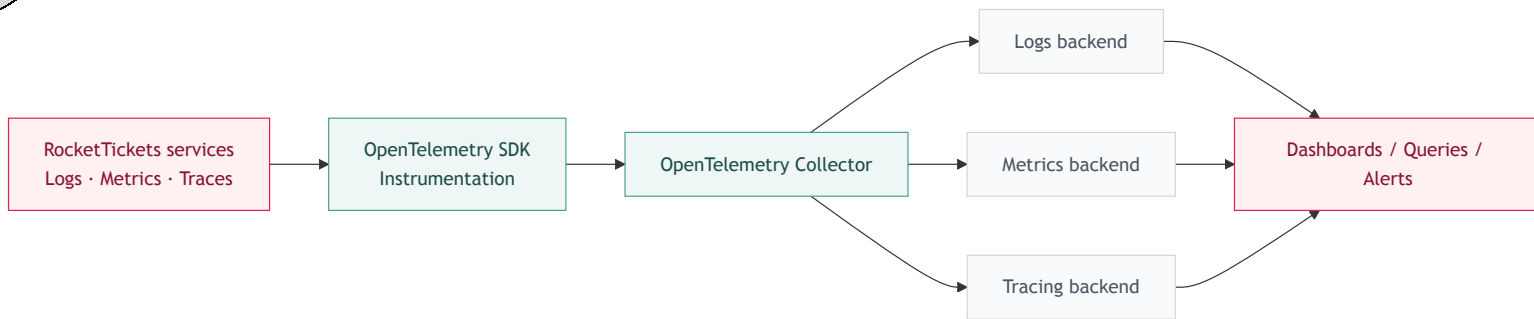
```
// OpenTelemetry auto-instrumentation on HTTP calls
// captures request bodies by default.

// A trace for POST /api/purchase now contains:
// {
//   "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
//   "cardToken": "tok_visa_4242",
//   "promoCode": "FRIEND20"
// }

// That promo code might identify who referred the user.
// Is that PII?
// Who decides?
// Where does it get stored, and for how long?
```

Auto-instrumentation is powerful. It's also indiscriminate. You opt into observability. You may not realise you're also opting into data collection.

Observability data has an architecture too



Privacy risk is not only in application code. It exists at every hop where telemetry is captured, transformed, stored, or queried.

Where do you redact sensitive data?

Layer	Strongest benefit	Main risk	Typical use
SDK / app	Data never leaves the service	Requires code changes across teams	Removing request bodies, headers, PII fields
Collector	Central policy for many services	If app emits raw data, collector becomes a critical trust point	Normalisation, masking, drop rules
Storage / backend	Easy to change without redeploying apps	Too late: sensitive data may already be stored or replicated	Retention policies, access control, field-level views

Earlier redaction gives stronger guarantees. Later redaction gives operational flexibility.

RocketTickets – minimum telemetry policy

Capture (allow)

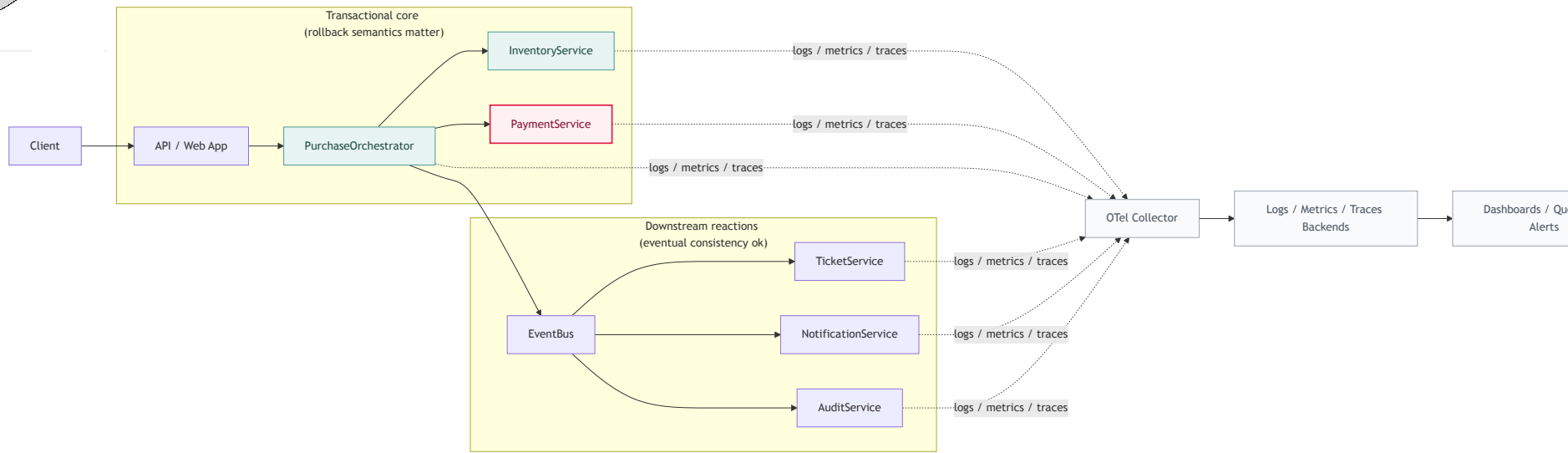
- Request route, method, status code, latency
- Service name and operation name
- Payment provider name (Stripe , PayPal)
- Business outcome flags (SeatReserved , PaymentConfirmed)
- Error category / exception type (*not full sensitive payloads*)

Do not capture (deny)

- Raw request/response bodies for checkout/payment flows
- Card tokens, card last4, emails, phone numbers
- user_id as a metric label/tag
- Promo/referral codes without explicit justification
- Free-form exception context that includes customer input

Policy is architecture: it defines what evidence exists during incidents and what liability you create while collecting it.

Flash sale incident – system view



Failure spike is visible in **PaymentService**, but the decision is architectural: *what extra telemetry can you add quickly without expanding privacy risk?*

Flash sale incident: what do you enable now?

Payment failures spike during a RocketTickets flash sale.

Ops asks for more telemetry to debug. Fast.

Legal warns against collecting anything that can identify customers.

Likely good moves (fast + safer)

- Error-rate metric by provider / endpoint
- Timeout / retry counters
- Span attributes for failure category (`timeout` , `declined` , `dependency_error`)
- Short-term sampling increase for failed requests only

Moves to challenge or refuse

- Logging raw payment request/response bodies
- Capturing card token / email to "correlate faster"
- Adding `user_id` as a metric label
- Dumping exception context that contains customer input

What additional telemetry do you enable in the next 10 minutes, and what do you explicitly refuse to collect?



The privacy tension

What do you redact, and at what layer?

In the SDK? In the collector? In the storage tier? Each has different guarantees and different failure modes.

Who decides what's PII in a system with five teams?

Marketing says promo codes are business data. Legal says they're personal data. Engineering says they need them to debug.

How do you balance "we need this to debug incidents" with "we shouldn't store this"?

Sampling? Masking? Short retention? Separate storage tiers with different access controls?

These questions don't have obvious answers. They are exactly what the assignment asks you to navigate.



Revisit

Block 3 – Home Reading

*Telemetry and observability terminology for the assignment and future labs.
Not presented live – acknowledged and pointed to.*



Revisit – Terminology

Observability and Telemetry

Observability – the ability to infer a system's internal state from its external outputs.

Telemetry – the data emitted by a system to support observation and diagnosis.

Signal – one category of telemetry data. In this course: **logs**, **metrics**, and **traces**.

Instrumentation – code or libraries that emit telemetry from an application.

Structured logging – logs emitted with fields (`trace_id` , `order_id` , `error_code`) rather than only free text.

Correlation – the ability to connect logs, metrics, and traces for the same request or incident.



Revisit – Traces and Spans

Trace – the end-to-end record of a request as it moves across services.

Span – one unit of work within a trace. A trace is made of one or more spans.

Root span – the first span in a trace (often the incoming HTTP request).

Child span – a span nested under another span (database call, external API call, internal operation).

Span attribute – key/value metadata attached to a span (for example `http.route` , `payment.provider` , `error.type`).

Span event – a timestamped event inside a span (for example "retry started", "fallback activated").



Revisit – Context Propagation

Context propagation – passing tracing context across process boundaries so downstream spans join the same trace.

Trace ID – identifier shared by all spans in the same trace.

Span ID – identifier of one specific span.

Parent Span ID – links a span to its parent in the trace tree.

Baggage – cross-service key/value context carried with the request. Powerful, but risky if used for personal data.

```
// Example (conceptual): enrich logs with current trace context
_logger.LogInformation(
    "Payment timeout. trace_id={TraceId} span_id={SpanId}",
    Activity.Current?.TraceId,
    Activity.Current?.SpanId);
```

Architectural caution: if context is not propagated, each service still emits telemetry – but the distributed trace becomes fragmented and diagnosis becomes guesswork.



Revisit – Metrics Concepts

Concept	Meaning	Example
Counter	Monotonic increasing count	<code>payment_failures_total</code>
Gauge	Value at a point in time	<code>queue_depth</code>
Histogram	Distribution of values	request latency buckets
Label / tag	Metric dimension for slicing	<code>provider=Stripe</code> , <code>endpoint=/purchase</code>
Cardinality	Number of unique label combinations	<code>user_id</code> labels explode cardinality

High-cardinality metrics are operationally expensive and often privacy-sensitive.

`user_id` , `email` , `promo_code` , and raw URLs are common mistakes.



Revisit – Logs in an Observability Stack

Log line – one recorded event.

Structured field – a queryable field inside a log (`order_id` , `error_code` , `trace_id`).

Retention – how long logs are stored.

Redaction – removing sensitive values before storage.

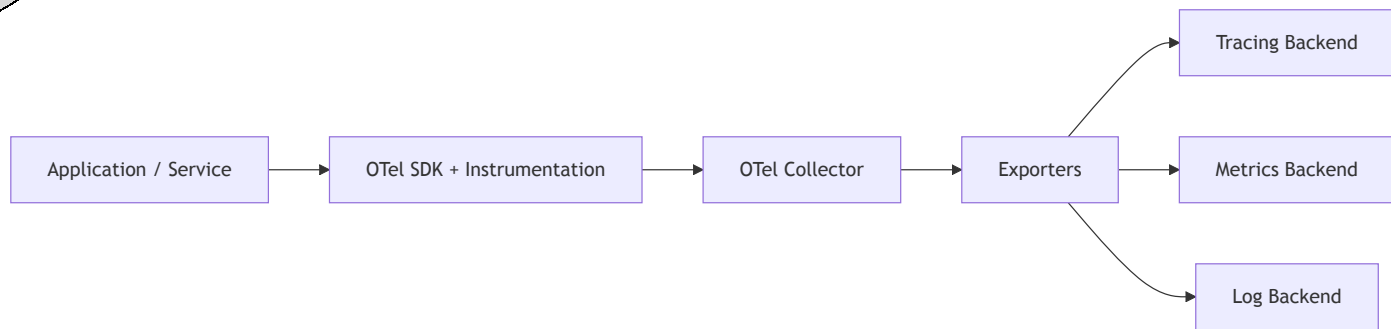
Masking – preserving shape while hiding value (for example `email=jo***@example.com`).

Log sampling – storing only a subset of logs under high volume (often by level, route, or outcome).

Practical rule: Logs should be useful for debugging without becoming a second database of customer data.



Revisit – OpenTelemetry Pipeline Terms



Auto-instrumentation – telemetry emitted by libraries/framework hooks with minimal code changes.

Manual instrumentation – telemetry emitted explicitly in application code (custom spans, attributes, events).

Collector – a process that receives, transforms, filters, and forwards telemetry.

Exporter – component that sends telemetry to a backend in a specific protocol/format.



Revisit – Sampling, Retention, and Privacy

Head sampling – sampling decision made at the start of a trace.

Tail sampling – sampling decision made after seeing the full trace (for example keep error traces).

PII (Personally Identifiable Information) – data that can identify a person directly or indirectly.

Data minimisation – collect only what is necessary for the stated purpose.

Access control – who can query telemetry, and at what level of detail.

Retention policy – how long each signal is kept (logs, traces, metrics may differ).

Architectural consequence: observability quality is not only about signal richness. It is also about controlled collection, controlled access, and controlled retention.



Revisit – Telemetry Design Checklist (RocketTickets)

- Can I debug the flash sale payment incident without logging customer identifiers?
- Are trace/span attributes allowlisted, or am I letting frameworks capture everything?
- Are metric labels bounded (no `user_id` , no free-text values)?
- Do logs include `trace_id` so I can correlate without copying payloads?
- Is redaction applied before storage for checkout/payment flows?
- Are error traces sampled differently from healthy traces?

Good telemetry is evidence with boundaries.



Revisit – References

OpenTelemetry

OpenTelemetry Documentation – Concepts (traces, metrics, logs, baggage, context propagation)
opentelemetry.io/docs/concepts/

OpenTelemetry Semantic Conventions – standard attribute names
opentelemetry.io/docs/specs/semconv/

Observability + Practice

Google SRE Book – Monitoring Distributed Systems (signal design and alerting tradeoffs)
sre.google/sre-book/monitoring-distributed-systems/

Course

Assignment 1 brief – use this terminology in your justification, not only in your implementation.